ASP.NET MVC

Complete Study Guide

Routing • Validation • Middleware • Identity • AJAX

Ahmed Zaher

9 Days — Full Course Notes with Diagrams & Examples

Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7 | Day 8 | Day 9

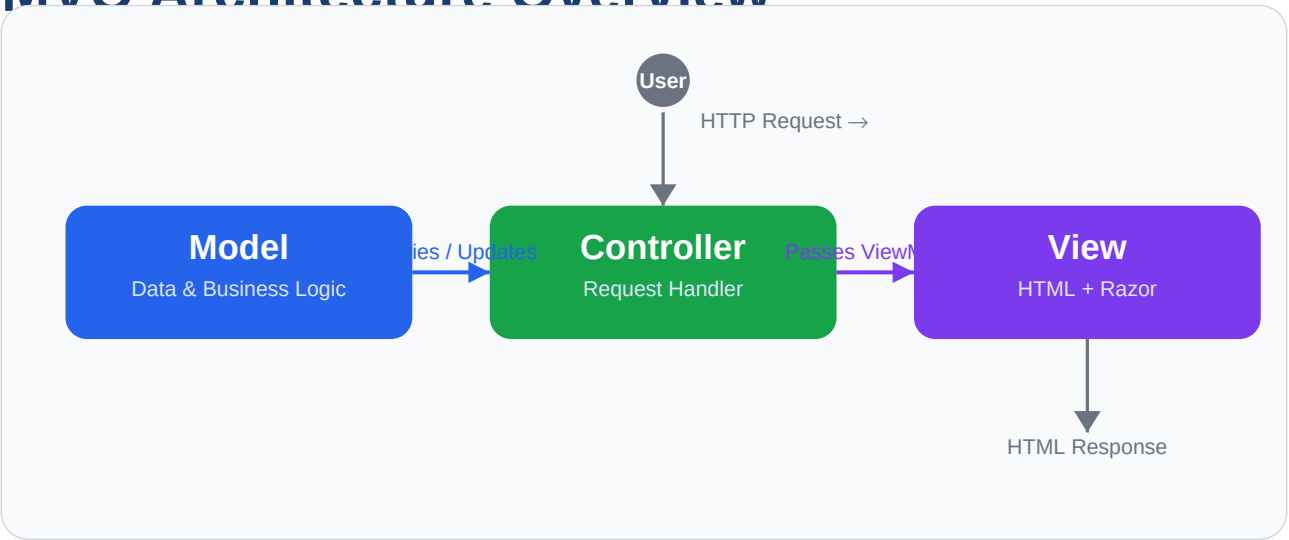
■ Table of Contents

DAY 1

Routing, Action Results & Razor Engine

How URLs map to Controllers and how Views are rendered

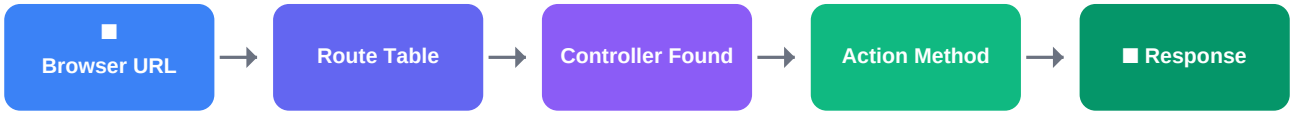
MVC Architecture Overview



ASP.NET MVC separates the application into three layers: **Model** (data & business logic), **Controller** (request handling), and **View** (HTML rendering). The user sends a request → the Controller processes it → queries the Model → passes data to the View → returns HTML.

1.1 Routing

Routing defines how incoming URLs are matched to Controllers and Actions. The default route pattern is: **{controller}/{action}/{id?}**



Example: URL /Employee/Details/5 → Controller: **EmployeeController** → Action: **Details(id=5)**

1.2 Action Result Types

Return Type	Class	Use Case
string / text	ContentResult	Return plain text
View	ViewResult	Render an HTML page
JSON data	JsonResult	AJAX / API calls
File download	FileResult	Send a file to browser
404 Not Found	NotFoundResult	Resource not found

401 Unauthorized	UnauthorizedResult	No access
Any of above	IActionResult (interface)	Flexible — can return any type

Code Example — Different Action Results

```C#
public ViewResult Show()
{
    ViewResult view = new();
    view.ViewName = "View1";
    return view;
}

public IActionResult Mix(int id)
{
    if (id % 2 == 0)
        return Content("Even number!");
    return View("View1");
}
```

■ **IActionResult** is the recommended return type when an action can return different result types (e.g., sometimes a View, sometimes JSON).

1.3 Razor Engine — C# inside HTML

Razor uses @ to switch from HTML to C#. It supports inline expressions, code blocks, conditionals, and loops.

```C#
@{
    int x = 10;
    int y = x + 10;
}

<!-- Inline expression -->
<h2>@x</h2>   <!-- Output: 10 -->

<!-- Condition -->
@if (x > y) {
    <h2>X is greater</h2>
} else {
    <h2>X = @x  Less than  y = @y</h2>
}

<!-- Loop -->
@for (int i = 0; i < 5; i++) {
    <li>@i</li>
}
```

DAY 2

Data Passing & State Management

ViewData · ViewBag · ViewModel · Session · Cookies · TempData

2.1 Three Ways to Pass Data to the View

When an Action needs to send data to a View, there are three mechanisms. **ViewModel** is always the recommended approach.

ViewData & ViewBag ■ Not Recommended

- Dictionary-based (object type)
- Requires casting on read
- No compile-time type safety
- Runtime errors go undetected
- Both share the same internal dictionary

ViewModel ■ Recommended

- Strongly-typed class
- No casting needed
- Compile-time checks
- IntelliSense support
- Easy to unit test

ViewData Example (Controller → View)

C#

```
// In Controller
ViewData["Name"] = "Ahmed";
ViewData["Age"]  = 25;
return View();

// In View
<h1>@ViewData["Name"]</h1>
<h2>@((int)ViewData["Age"] + 5)</h2>  // Cast needed!
```

ViewModel Example (Full Flow)

```C#
// 1. Define the ViewModel class
public class EmployeeViewModel
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public List<string> Branches { get; set; }
}

// 2. Controller fills and sends it
public IActionResult Details(int id)
{
    var vm = new EmployeeViewModel
    {
        Name = "Ahmed", Salary = 10000,
        Branches = new() { "Cairo", "Alex" }
    };
    return View(vm);
}

// 3. View uses it with @model
@model EmployeeViewModel
<h1>@Model.Name</h1>
<h2>Salary: @Model.Salary</h2>
```

2.2 State Management Comparison

Because the Controller object is created and destroyed with each request, we need special mechanisms to persist data.

Feature	TempData	Session	Cookies
Storage	Session (server)	Server memory	Client browser
Lifetime	1 request (+ 1 with Keep)	20 min idle (configurable)	Based on Expires property
Survives Redirect?	■ Yes	■ Yes	■ Yes
Best Use	Flash messages between actions	User data during browsing session	Persistent preferences (remember me)
Type Safety	■ Object	■ String / Int32	■ String only

Session — Configuration & Usage

C#

```
// Program.cs — register session service
builder.Services.AddSession(options => {
    options.IdleTimeout = TimeSpan.FromMinutes(30);
});
app.UseSession();  // Add middleware

// Controller — set session values
HttpContext.Session.SetString("Name", "Ahmed");
HttpContext.Session.SetInt32("Age", 25);

// Controller — read session values
string name = HttpContext.Session.GetString("Name");
int? age  = HttpContext.Session.GetInt32("Age");
```

Cookies — Set & Read

C#

```
public IActionResult SetCookie()
{
    CookieOptions opts = new CookieOptions();
    opts.Expires = DateTime.Now.AddDays(7);
    HttpContext.Response.Cookies.Append("Name", "Ahmed");
    HttpContext.Response.Cookies.Append("Age", "25", opts);
    return Content("Cookie saved!");
}

public IActionResult GetCookie()
{
    string name = HttpContext.Request.Cookies["Name"];
    return Content($"Hello {name}");
}
```

DAY 3

Model Binding & Form Handling

Query strings · Route params · Form POST · Redirects

3.1 What is Model Binding?

Model Binding is the automatic process of mapping HTTP request data (URL query strings, route parameters, form fields) into Action method parameters.

3.2 URL Binding Sources

Source	URL Example	Priority	Code
Query String	/Emp?name=Ahmed&age=20	Low	Request.QueryString
Route Parameter	/Emp/Details/10	High	Request.RouteValues
Form POST	<form method='post'>	Medium	Request.Form
Request Body (API)	JSON payload	High	[FromBody]

■■ **RouteValues take priority over QueryString. If both contain the same parameter, the route value wins.**

Code Examples

C#

```
// Query String: /Bind/Emp?name=Ahmed&age=20
public IActionResult Emp(string name, int age)
{
    return Content($"Name={name}, Age={age}");
}

// Route Param: /Bind/Emp/10
public IActionResult Details(int id)
{
    return Content($"ID={id}");
}

// Object binding: /Bind/Emp?id=1&name=Ahmed&salary=9000
public IActionResult SaveEmp(Employee emp)
{
    // emp.Id=1, emp.Name='Ahmed', emp.Salary=9000 — auto-mapped!
    return View(emp);
}
```

3.3 HttpGet vs HttpPost

C#

```
[HttpGet]  // handles GET (display form)
public IActionResult Edit(int id)
{
    var emp = repo.GetById(id);
    return View(emp);
}

[HttpPost]  // handles POST (save form)
public IActionResult Edit(Employee emp)
{
    if (ModelState.IsValid)
    {
        repo.Update(emp);
        return RedirectToAction("Index");
    }
    return View(emp);
}
```

DAY 4

HTML Helpers, Tag Helpers & Layout

Generating forms · RenderBody · RenderSection

4.1 Three Ways to Build Forms

Method	Syntax	Type-safe?	Recommended?
Raw HTML	<input type='text' name='Name'>	■ No	■ Old way
HTML Helper	@Html.TextBoxFor(m => m.Name)	■ Yes	■■ Legacy
Tag Helper	<input asp-for='Name'>	■ Yes	■ Modern

Tag Helper Form — Full Example

C#

```
@model UserModel

<form method='post' asp-action='Save' asp-controller='Account'>

    <div class='form-group'>
        <label asp-for='Name'></label>  <!-- auto label text -->
        <input asp-for='Name' class='form-control' />
        <span asp-validation-for='Name' class='text-danger'></span>
    </div>

    <div class='form-group'>
        <label asp-for='Password'></label>
        <input asp-for='Password' class='form-control' />
        <span asp-validation-for='Password' class='text-danger'></span>
    </div>

    <button type='submit' class='btn btn-primary'>Submit</button>
</form>

@section Scripts {
    <partial name='_ValidationScriptsPartial' />
}
```

■ **asp-for='Name'** automatically sets: **name='Name'**, **id='Name'**, **value='@Model.Name'** and wires up client-side validation.

4.2 Layout, RenderBody & RenderSection

C#

```
<!-- _Layout.cshtml — the master template -->
<!DOCTYPE html>
<html><head>
    <title>@ViewData["Title"]</title>
</head><body>
    <header>Shared Navigation Here</header>

    @RenderBody()  <!-- View content injected here -->

    @RenderSection("Sidebar", required: false)
    <footer>Shared Footer</footer>
</body></html>

<!-- In a specific View — override layout & define section -->
@{
    Layout = "NewLayout";  // or null to disable layout
}
@section Sidebar {
    <p>Sidebar content here</p>
}
```

DAY 5

Validation

Data Annotations · Custom · Remote · CSRF Protection

5.1 Validation Layers

5.2 Common Data Annotations

Annotation	Example	Description
[Required]	[Required(ErrorMessage='Field required')]	Field must not be empty
[StringLength]	[StringLength(50, MinimumLength=2)]	String length range
[EmailAddress]	[EmailAddress]	Valid email format
[MinLength]	[MinLength(6)]	Minimum characters
[Range]	[Range(18, 100)]	Number within range
[Compare]	[Compare('Password')]	Two fields must match
[RegularExpression]	[RegularExpression(@'\w+\.(jpg|png)')]	Must match regex pattern
[DataType]	[DataType(DataType.Password)]	Affects rendering type

Full Validation Example

C#

```
// Model — rules defined here
public class RegisterViewModel
{
    [Required(ErrorMessage = "Username required")]
    [StringLength(50, MinimumLength = 3)]
    public string Username { get; set; }

    [Required][EmailAddress]
    public string Email { get; set; }

    [Required][MinLength(6)]
    public string Password { get; set; }

    [Compare("Password", ErrorMessage = "Passwords don't match")]
    public string ConfirmPassword { get; set; }

    [Range(18, 100, ErrorMessage = "Age 18-100")]
    public int Age { get; set; }
}

// Controller — check ModelState
[HttpPost]
public IActionResult Register(RegisterViewModel model)
{
    if (!ModelState.IsValid)
        return View(model);    // back to form with errors
    return RedirectToAction("Success");
}
```

5.3 CSRF Protection — [ValidateAntiForgeryToken]

Cross-Site Request Forgery (CSRF) tricks a logged-in user into submitting a malicious form. ASP.NET Core protects against this by generating a hidden token in every form.

C#

```
// Controller — apply to every POST action
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Save(UserModel model) { ... }

// Tag Helper form — token added AUTOMATICALLY
<form asp-action='Save' method='post'>  <!-- auto token -->
    ...
</form>

// HTML Helper form — token must be MANUAL
@using (Html.BeginForm('Save', 'Home', FormMethod.Post))
{
    @Html.AntiForgeryToken()  // required!
    ...
}
```

■■ **Without the token, the request is rejected with HTTP 403 Forbidden. Always use [ValidateAntiForgeryToken] on every POST action that modifies data.**

5.4 Remote Validation

C#

```
// Model — triggers AJAX call on field change
[Remote(action: "CheckEmail", controller: "Account", ErrorMessage = "Email taken!")]
public string Email { get; set; }

// Controller — returns JSON true/false
public IActionResult CheckEmail(string Email)
{
    bool exists = db.Users.Any(u => u.Email == Email);
    return Json(!exists);  // true = valid, false = error
}
```

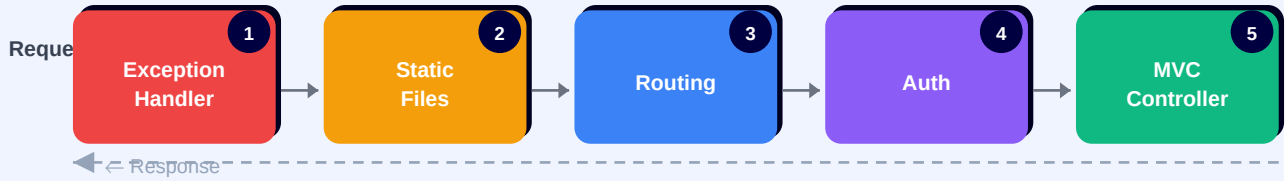
DAY 6

Middleware

Request pipeline · Built-in middleware · Authentication vs Authorization

6.1 Middleware Pipeline

Middleware components form a pipeline that every HTTP request flows through. Each component can process the request, pass it to the next component, or short-circuit the pipeline.



6.2 Three Types of Middleware

Type	Method	Behavior
Execute & Continue	app.Use()	Runs code, then calls next middleware
Terminal	app.Run()	Runs code, stops the pipeline
Branch by URL	app.Map()	Branches based on URL path

Custom Middleware Example

```C#
// Execution order: M1 → M2 → Terminal → M2 → M1
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("1) Before\n");
    await next.Invoke();  // call next middleware
    await context.Response.WriteAsync("1) After\n");
});

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("2) Before\n");
    await next.Invoke();
    await context.Response.WriteAsync("2) After\n");
});

app.Run(async (context) =>  // terminal — no next
{
    await context.Response.WriteAsync("3) Terminal\n");
});
```

Output order: ***1) Before → 2) Before → 3) Terminal → 2) After → 1) After***

6.3 Authentication vs Authorization

Authentication (UseAuthentication)

- Answers: WHO are you?
- Checks cookie / JWT token
- Fills HttpContext.User
- Must come BEFORE Authorization
- Methods: Password, JWT, OAuth

Authorization ([Authorize])

- Answers: WHAT can you do?
- Checks roles and policies
- Returns 403 if permission denied
- Runs AFTER Authentication
- Attributes: [Authorize], [AllowAnonymous]

DAY 7

Repository Pattern, SOLID & Dependency Injection

Clean architecture · IoC Container · Service lifetimes

7.1 SOLID Principles

S **Single Responsibility**
Class => one job only

O **Open / Closed**
Extend, never modify

L **Liskov Substitution**
Subclass works as parent

I **Interface Segregation**
Small focused interfaces

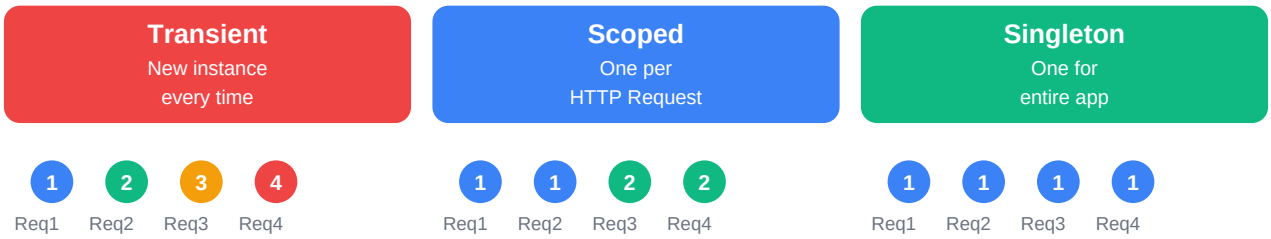
D **Dependency Inversion**
Depend on abstractions

7.2 Repository Pattern

C#

```
// 1. Interface — defines the contract
public interface IEmployeeRepository
{
    List<Employee> GetAll();
    Employee GetById(int id);
    void Add(Employee obj);
    void Update(Employee obj);
    void Delete(int id);
    void Save();
}

// 2. Implementation — talks to DB
public class EmployeeRepository : IEmployeeRepository
{
    private readonly ITIContext _context;
    public EmployeeRepository(ITIContext context)
        => _context = context;

    public List<Employee> GetAll() => _context.Employee.ToList();
    public Employee GetById(int id) => _context.Employee.FirstOrDefault(e => e.Id == id);
    public void Add(Employee obj) => _context.Add(obj);
    public void Save() => _context.SaveChanges();
}

// 3. Controller — uses interface, NOT concrete class
public class EmployeeController : Controller
{
    private readonly IEmployeeRepository _repo;
    public EmployeeController(IEmployeeRepository repo)
        => _repo = repo;  // injected by DI container
}
```

7.3 Dependency Injection — Service Lifetimes



C#

```
// Program.cs — Register services
builder.Services.AddDbContext<ITIContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("cs")));

// Scoped = one per request (best for repositories)
builder.Services.AddScoped<IEmployeeRepository, EmployeeRepository>();
builder.Services.AddScoped<IDepartmentRepository, DepartmentRepository>();

// appsettings.json connection string:
// "ConnectionStrings": { "cs": "Data Source=.;Initial Catalog=ITI;..." }
```

DAY 8

Filters & ASP.NET Core Identity

Action Filters · Exception Filters · Users · Roles · Claims

8.1 Filters Pipeline

Request Pipeline (MVC Level)

Each filter runs before/after the action method

8.2 Custom Action Filter

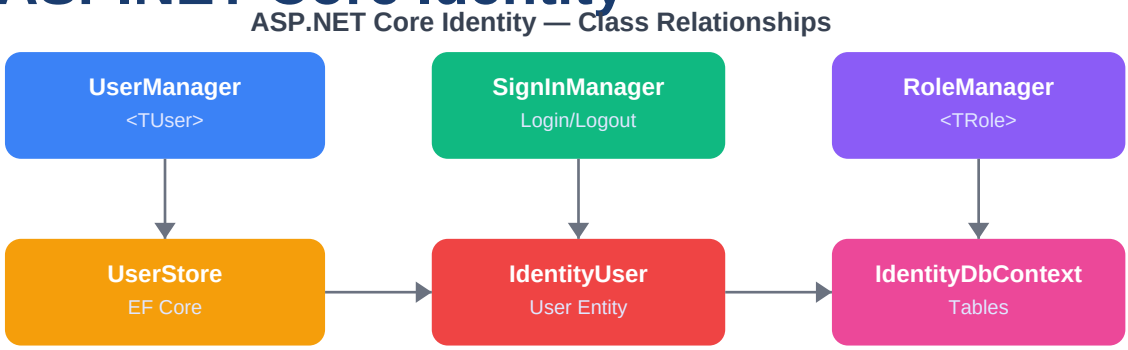
C#

```
public class LogActionFilter : Attribute, IActionFilter
{
    // Runs BEFORE the action method
    public void OnActionExecuting(ActionExecutingContext ctx)
    {
        Console.WriteLine($"[Before] {ctx.ActionDescriptor.DisplayName}");
    }

    // Runs AFTER the action method
    public void OnActionExecuted(ActionExecutedContext ctx)
    {
        Console.WriteLine($"[After] {ctx.ActionDescriptor.DisplayName}");
    }
}

// Usage on Controller or Action
[LogActionFilter]
public IActionResult Index() { return View(); }
```

8.3 Exception Filter

```C#
public class GlobalExceptionFilter : Attribute, IExceptionFilter
{
    public void OnException(ExceptionContext ctx)
    {
        string msg = ctx.Exception.Message;
        ctx.Result = new ViewResult { ViewName = "Error" };
        ctx.ExceptionHandled = true;  // stop exception propagating
    }
}

// Apply globally in Program.cs
builder.Services.AddControllersWithViews(options =>
    options.Filters.Add(new GlobalExceptionFilter()));
```

8.4 ASP.NET Core Identity



Setup Steps

Step	Action
1	Install Microsoft.AspNetCore.Identity.EntityFrameworkCore
2	Create ApplicationUser : IdentityUser (add custom properties)
3	ITIContext : IdentityDbContext<ApplicationUser>
4	Add-Migration + Update-Database
5	Create AccountController (inject UserManager + SignInManager)
6	Create RegisterViewModel + LoginViewModel
7	Register Identity in Program.cs

Register & Login Code

C#

```
// Register
[HttpPost] public async Task<IActionResult> SaveRegister(RegisterViewModel vm)
{
    if (!ModelState.IsValid) return View('Register', vm);
    ApplicationUser user = new() { UserName = vm.UserName, Address = vm.Address };
    IdentityResult result = await userManager.CreateAsync(user, vm.Password);
    if (result.Succeeded) {
        await signInManager.SignInAsync(user, isPersistent: false);
        return RedirectToAction("Index");
    }
    foreach (var err in result.Errors) ModelState.AddModelError('', err.Description);
    return View('Register', vm);
}

// Login
[HttpPost] public async Task<IActionResult> SaveLogin(LoginViewModel vm)
{
    var user = await userManager.FindByNameAsync(vm.Name);
    if (user != null) {
        bool ok = await userManager.CheckPasswordAsync(user, vm.Password);
        if (ok) { await signInManager.SignInAsync(user, vm.RememberMe);
              return RedirectToAction("Dashboard"); }
    }
    ModelState.AddModelError('', 'Invalid username or password');
    return View('Login', vm);
}

// Program.cs — register identity
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(opts => {
    opts.Password.RequiredLength = 6;
    opts.Password.RequireNonAlphanumeric = false;
}).AddEntityFrameworkStores<ITIContext>();
```

DAY 9

Partial Views, AJAX & Routing Deep Dive

Reusable UI · Async updates · Convention & Attribute routing

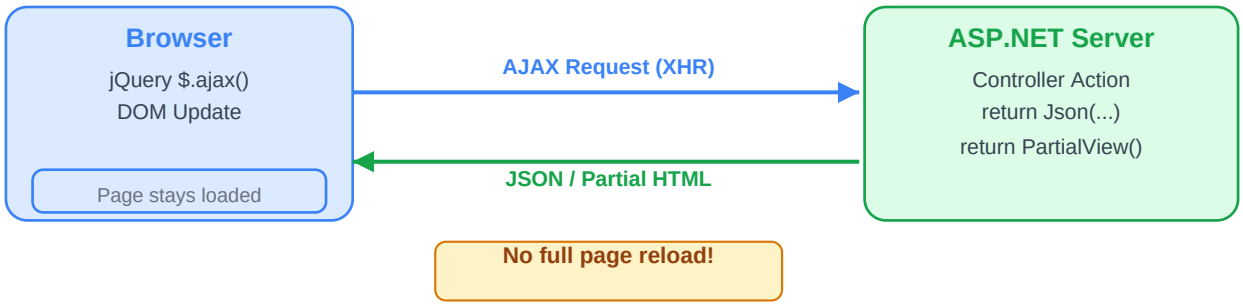
9.1 Partial Views

A Partial View is a small, reusable Razor view injected into a parent view. Name convention: prefix with _ (e.g. _**EmpCard.cshtml**). It does NOT include its own layout.

C#

```
// _EmpCard.cshtml — the partial view
@model Employee
<div class='card'>
    <h2>@Model.Name</h2>
    <p>Salary: @Model.Salary</p>
    <p>Dept: @Model.DepartmentID</p>
</div>

// Parent View — include the partial
<partial name='_EmpCard' />                    // inherit parent model
<partial name='_EmpCard' model='Model.Emp' />   // pass different model

// Or with Html Helper:
@await Html.PartialAsync('_EmpCard', Model.Emp)

// Controller can return partial view for AJAX:
public IActionResult EmpCard(int id)
    => PartialView('_EmpCard', repo.GetById(id));
```

9.2 AJAX Request Flow



AJAX with jQuery — Practical Example

C#

```
<!-- Parent view — click loads employee card without page reload -->
<div id='empCard'></div>
<a href='#' onclick='loadCard(@item.Id)'>Details</a>

<script src='~/lib/jquery/dist/jquery.min.js'></script>
<script>
function loadCard(empId) {
    event.preventDefault();  // stop default link navigation
    $.ajax({
        url: '/Employee/EmpCard/' + empId,
        success: function(result) {
            $('#empCard').html(result);  // inject partial HTML
        }
    });
}
</script>

// Cascading Dropdowns — load employees when dept changes
function loadEmps() {
    var deptId = document.getElementById('DeptId').value;
    $.ajax({
        url: '/Dept/GetEmps?deptId=' + deptId,
        success: function(data) {
            var sel = document.getElementById('EmpList');
            sel.innerHTML = '';
            for (let emp of data) {
                sel.innerHTML += `<option value='${emp.id}'>${emp.name}</option>`;
            }
        }
    });
}
```

9.3 Routing Deep Dive

Convention-Based vs Attribute Routing

Convention-Based (MVC)

- Defined in Program.cs
- Pattern: {controller}/{action}/{id?}
- Applies to all controllers
- Great for CRUD applications
- Less repetition

Attribute Routing (API)

- Defined per controller/action
- [Route('api/products/{id:int}')]
- Fine-grained URL control
- Great for REST APIs
- More explicit and readable

Route Constraints

Constraint	Example	Matches
:int	{id:int}	Integers only: 1, 42
:alpha	{name:alpha}	Letters only: abc, Ahmed

:bool	{active:bool}	true or false
:min(n)	{age:min(18)}	Integer >= 18
:max(n)	{age:max(100)}	Integer <= 100
:range(a,b)	{score:range(0,100)}	Integer between 0 and 100
:length(n)	{code:length(5)}	String of exactly 5 chars
:regex(pat)	{zip:regex(^\d{5}$)}	Matches regex pattern

C#

```
// Convention-based in Program.cs
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");

// Custom route — /products/list maps to Products/AllItems
app.MapControllerRoute("custom", "products/list",
    new { controller = "Products", action = "AllItems" });

// With constraints: /report/2024 (year must be int)
app.MapControllerRoute("report", "report/{year:int}",
    new { controller = "Report", action = "Index" });

// Attribute Routing (great for APIs)
[Route("api/[controller]")]
public class ProductsController : Controller
{
    [Route("{id:int}")]
    public IActionResult Details(int id) { ... }
}
```

Quick Reference — All 9 Days

Day	Topic	Key Concepts
1	Routing & Views	IActionResult types, Razor @syntax, URL → Controller mapping
2	Data Passing	ViewData/ViewBag/ViewModel, Session, Cookies, TempData
3	Model Binding	Query string, route params, form POST, [HttpGet]/[HttpPost]
4	HTML/Tag Helpers	asp-for, asp-action, RenderBody, RenderSection, Layout
5	Validation	Data Annotations, ModelState, Remote, [ValidateAntiForgeryToken]
6	Middleware	Use/Run/Map, pipeline order, Authentication vs Authorization
7	Repository + SOLID	Repository pattern, SOLID principles, DI Container, lifetimes
8	Filters + Identity	Action/Exception filters, UserManager, SignInManager, Roles, Claims
9	Partial + AJAX	Partial views, $.ajax(), JSON, cascading dropdowns, routing constraints

Good Luck! ■

Ahmed Zaher — ASP.NET MVC Complete Study Guide